

Homework 05

Set a random seed before each simulation question and report it. You may use standard packages unless a question asks you to write the calculation yourself. Use equal neighbor weights throughout. Here p counts the covariates used to measure distance; do not add an intercept column to a KNN predictor matrix. R and Python use different random-number generators, so their exact simulation results need not agree.

Question 1: Estimate the bias and variance of KNN

In our previous homework, we used repeated simulations to study the behavior of linear regression estimators. We will now use the same idea for KNN. Consider the regression model

$$Y_i = f(\mathbf{X}_i) + \epsilon_i, \quad f(\mathbf{x}) = 0.5x_1 + \sin(x_2) - 0.3x_3^2,$$

where the three covariates are independent standard normal variables and $\epsilon_i \sim \mathcal{N}(0, 0.1^2)$ independently of the covariates and other observations. Generate $n = 400$ training observations. Our goal is to estimate the mean response at

$$\mathbf{x}_0 = (0.5, 0.7, 1)^\top.$$

Use Euclidean distance and consider $k = 1, 3, 5, \dots, 29$.

- Calculate $f(\mathbf{x}_0)$. Generate one training dataset and obtain the KNN prediction at $\mathbf{x}_0$ for each value of k .
- Independently repeat the generation of the training covariates and responses 200 times. Within each repetition, use the same training data for every value of k . Store the predictions in a matrix with 200 rows and one column for each value of k . Use this matrix to estimate the bias, variance, and mean squared error of the estimated mean response. Use divisor 200 when calculating the empirical variance, and verify numerically that

$$\text{mean squared error} = \text{squared bias} + \text{variance}.$$

Plot squared bias, variance, and mean squared error against k in one figure. Explain the pattern and compare it with the bias-variance trade-off for ridge regression or lasso.

Answer

Part a

```
Seed for Question 1(a): 5101
f(x0) = 0.594217687237691
```

KNN predictions for Question 1(a):

```
k = 1: 0.246652
k = 3: 0.312294
k = 5: 0.363221
k = 7: 0.340096
k = 9: 0.307758
k = 11: 0.393422
k = 13: 0.345718
k = 15: 0.370719
k = 17: 0.332118
k = 19: 0.373952
k = 21: 0.345101
k = 23: 0.364689
k = 25: 0.342802
k = 27: 0.318165
k = 29: 0.335597
```

Part b

Seed for Question 1(b): 5102

Results:

k	Mean Pred	Bias	Bias^2	Variance	MSE	Bias^2+Var
1	0.594449	0.000232	0.000000	0.044842	0.044842	0.044842
3	0.575394	-0.018824	0.000354	0.022621	0.022975	0.022975
5	0.564157	-0.030061	0.000904	0.016391	0.017294	0.017294
7	0.549453	-0.044764	0.002004	0.013070	0.015074	0.015074
9	0.537696	-0.056522	0.003195	0.010332	0.013527	0.013527
11	0.528756	-0.065462	0.004285	0.009113	0.013398	0.013398
13	0.520482	-0.073735	0.005437	0.008017	0.013454	0.013454
15	0.509874	-0.084343	0.007114	0.007711	0.014824	0.014824
17	0.498933	-0.095285	0.009079	0.007262	0.016341	0.016341
19	0.498476	-0.095742	0.009167	0.007106	0.016272	0.016272
21	0.495628	-0.098590	0.009720	0.006894	0.016614	0.016614
23	0.486434	-0.107784	0.011617	0.007312	0.018930	0.018930
25	0.480931	-0.113287	0.012834	0.006847	0.019680	0.019680
27	0.475588	-0.118629	0.014073	0.006443	0.020515	0.020515
29	0.469474	-0.124744	0.015561	0.006367	0.021928	0.021928

Maximum absolute difference between MSE and squared bias + variance:
7.806255641895632e-17

Decomposition verified: True

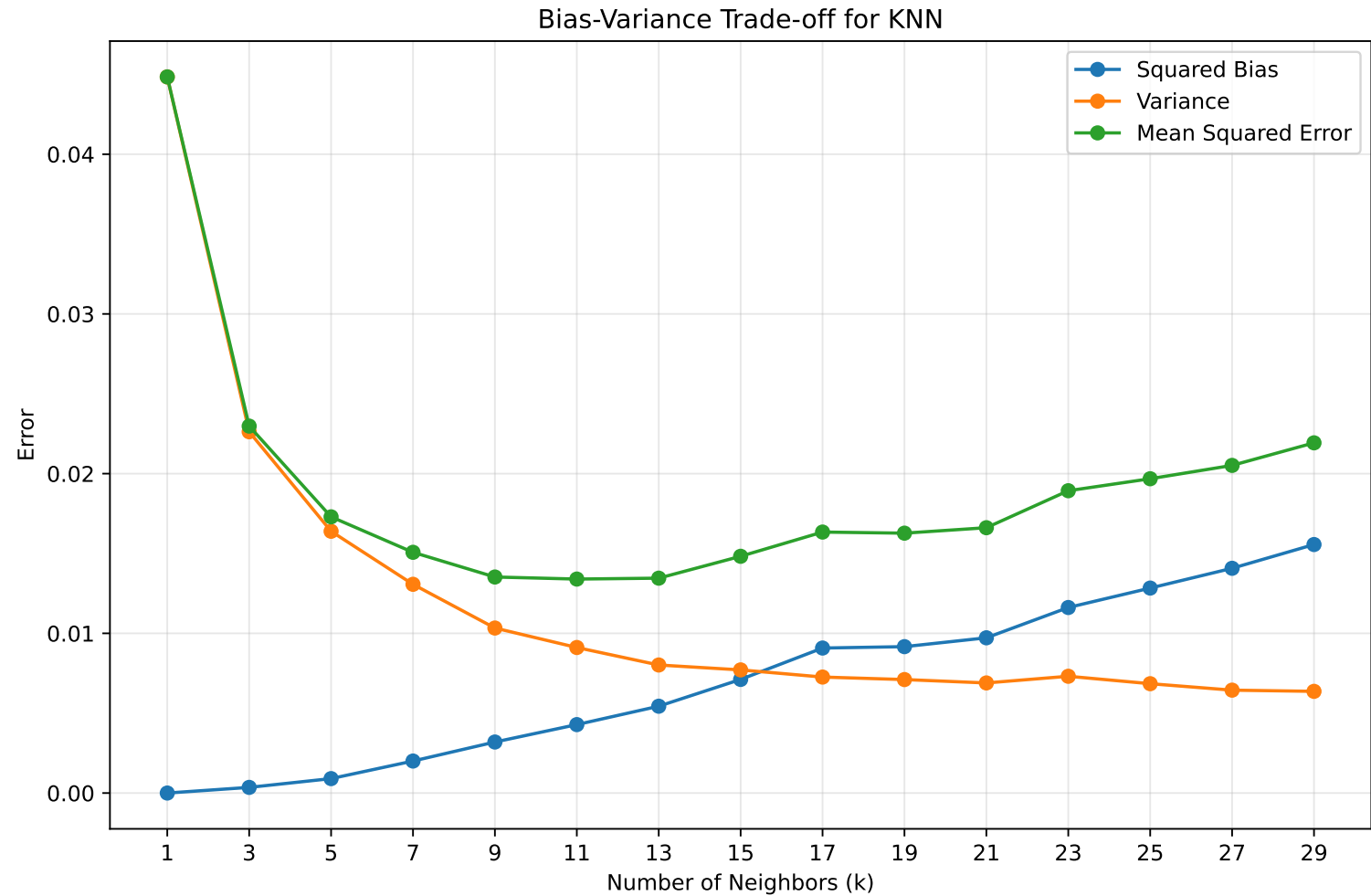


Figure 1: Squared bias, variance, and MSE of the KNN estimate of $f(\mathbf{x}_0)$ across k (200 repetitions).

As k increases, the variance decreases because the prediction is averaged over more neighbors, making KNN less sensitive to changes in the training data. However, the extra smoothing also increases the bias. The mean squared error reflects both components,

$$\text{MSE} = (\text{Bias})^2 + \text{Variance}.$$

The MSE decreases at first and reaches its lowest value around $k = 11$, with an MSE of about 0.0134. The MSE curve is nearly flat from $k = 9$ to $k = 13$ (0.01353, 0.01340, and 0.01345), so $k = 11$ is the minimum by only a small margin. After that, the increase in squared bias becomes larger than the reduction in variance, so the MSE starts to increase. The numerical results also verify the MSE decomposition.

This pattern is similar to ridge or lasso regression. Increasing the penalty in ridge or lasso generally reduces variance but increases bias. Similarly, increasing k in KNN increases smoothing, which lowers variance but increases bias.

Question 2: Compare KNN with lasso

Use the response model from Question 1, including the noise standard deviation 0.1, but generate $p = 30$ covariates. The response still depends only on the first three. Consider these two settings:

- **Setting 1:** all 30 covariates are independent standard normal variables.
- **Setting 2:** the covariates have a multivariate normal distribution with mean zero and covariance matrix Σ , where

$$\Sigma_{jj} = 1, \quad \Sigma_{jl} = 0.8 \quad \text{for } j \neq l.$$

In each setting, generate 400 training observations and 1,000 independent test observations. Fit 5NN using Euclidean distance on the supplied covariate scale. All covariates already have population variance one.

For lasso, use a linear model containing the 30 covariates and a separately fitted, unpenalized intercept. Follow the Week 4 objective,

$$L_{\lambda}(\beta) = \frac{1}{2n} \|\tilde{\mathbf{y}} - \mathbf{X}\beta\|_2^2 + \lambda\|\beta\|_1,$$

where $\mathbf{X} \in \mathbb{R}^{n \times 30}$ contains the centered and standardized covariates and $\tilde{\mathbf{y}}$ is the centered training response. Select the penalty by ten-fold cross-validation over 41 logarithmically spaced values from 10^{-4} to 1. Estimate all means and scales using only the training portion of each fold, with divisor equal to that portion’s sample size. After selecting the penalty, refit on all training observations and apply the fitted transformations unchanged to the test observations. Do not add nonlinear terms to the lasso model.

- a. Fit both methods in each setting. Use the same training and test data for the two methods within a setting. Report their test MSE values in one table, together with the selected lasso penalty and the number of nonzero lasso slopes. You only need to generate one dataset for each setting; repeated simulations are not required.
- b. Compare the results between the two settings. Explain how correlation can change the usefulness of Euclidean neighborhoods, and why lasso does not use all covariates in the same way as KNN.
- c. The response model contains $\sin(X_2)$ and X_3^2 , but the lasso fit uses only linear terms. Explain how this difference affects the comparison. Would the results justify a general claim that correlation always helps KNN, or that one method is always better? Explain why.

Answer

Part a

Seed for Setting 1: 5201
Seed for Setting 2: 5202

Question 2(a) Results:

Setting	5NN Test MSE	Lasso Test MSE	Selected Lambda	Nonzero Lasso Slopes
Setting 1: Independent	0.550868	0.260395	0.019953	13
Setting 2: Correlated	0.198594	0.320550	0.019953	7

The random seeds were 5201 for Setting 1 and 5202 for Setting 2. In the independent setting, lasso had a lower test MSE than 5NN (0.260 vs. 0.551). In the correlated setting, the result reversed, with 5NN having the lower test MSE (0.199 vs. 0.321). The selected penalty was $\lambda = 0.019953$ in both settings, while lasso selected fewer nonzero slopes in the correlated setting (7 vs. 13).

Part b

The correlation had a large effect on KNN. With 30 independent covariates, Euclidean distance is affected by all the irrelevant variables, which makes it harder for KNN to find truly similar observations. When the covariates are highly correlated, they tend to move together, so the neighborhoods can provide more useful information about the variables related to the response. This helps explain why the 5NN MSE dropped from 0.551 to 0.199.

Lasso handles irrelevant variables differently because it can shrink their coefficients toward zero instead of using every variable equally. It kept 13 nonzero slopes in Setting 1 but only 7 in Setting 2.

Part c

The true response contains nonlinear terms, $\sin(X_2)$ and X_3^2 , while the lasso model uses only linear terms. Therefore, lasso cannot fully represent the true relationship. KNN does not assume a linear form, so it can capture nonlinear patterns through local neighborhoods.

However, these results do not mean that correlation always helps KNN or that one method is always better. Lasso performed better in Setting 1, while 5NN performed better in Setting 2. The performance of each method depends on the correlation structure, irrelevant covariates, sample size, and the form of the true relationship.

Question 3: Observed dimension and latent dimension

The lecture's handwritten digit example shows that KNN can work with many measured covariates. We will use a simulation to investigate why the structure of those covariates matters.

Let $p = 100$ be the number of observed covariates and let m be the number of latent variables. For each observation, generate

$$\mathbf{X}_i = \mathbf{A}\mathbf{Z}_i + \eta_i, \quad Y_i = Z_{i1} + Z_{i2} + \epsilon_i,$$

where $\mathbf{A} \in \mathbb{R}^{100 \times m}$ has independent entries from $\text{Uniform}[-2, 2]$. The entries of $\mathbf{Z}_i \in \mathbb{R}^m$ and $\eta_i \in \mathbb{R}^{100}$, and the scalar ϵ_i , are independent standard normal variables, also independent of $\mathbf{A}$. Generate $\mathbf{A}$ once and use it for all training and test observations in that dataset.

Compare $m = 3$ and $m = 30$. In each setting, generate 200 training observations and 200 independent test observations. Fit KNN using only the 100 observed covariates in $\mathbf{X}$, with Euclidean distance on their generated scale. The latent variables in $\mathbf{Z}$ are used to generate the data but are not available to the fitted model. Consider $k = 2, 10, 18, \dots, 82$.

- Generate one dataset for each value of m . Fit KNN for every value of k and calculate test MSE. Within a setting, use the same training and test observations for all values of k .
- Repeat the complete experiment 50 times, generating a new $\mathbf{A}$ and new observations in each repetition. Plot average test MSE against k , with one curve for each value of m . These curves compare the specified choices of k in a controlled simulation; they are not a cross-validation procedure for selecting a final model.
- Both settings have 100 observed covariates, and the response depends on only two latent variables. Explain why their KNN performance can still differ. What role does the noise η_i play, and why are the observed data not confined exactly to an m -dimensional linear space?

Answer

Part a

k values: [2 10 18 26 34 42 50 58 66 74 82]

Seed for Question 3(a): 5301

Question 3(a):

k	MSE (m=3)	MSE (m=30)
2	1.889906	3.260898
10	1.359737	2.399233
18	1.319830	2.268390
26	1.374949	2.302415
34	1.470738	2.297222
42	1.545327	2.290948
50	1.614722	2.329466
58	1.688957	2.358399
66	1.747968	2.375849
74	1.831940	2.377026
82	1.905744	2.414425

The random seed used for Part (a) was 5301. For both $m = 3$ and $m = 30$, the test MSE decreased as k increased from 2 and then started increasing again. For this single dataset, the lowest test MSE occurred at $k = 18$ for both settings. The minimum MSE was about 1.320 for $m = 3$ and 2.268 for $m = 30$.

Part b

Seed for Question 3(b): 5302

Question 3(b): Average over 50 repetitions

k	Average MSE (m=3)	Average MSE (m=30)
2	1.704760	3.067225
10	1.297739	2.302969
18	1.301599	2.304089
26	1.340366	2.328722
34	1.394410	2.356216
42	1.449297	2.382087
50	1.506022	2.412458
58	1.558366	2.434853
66	1.615170	2.463625
74	1.674301	2.493065
82	1.733905	2.520916

Lowest average MSE among specified k values:

m = 3: k = 10, average MSE = 1.297739

m = 30: k = 10, average MSE = 2.302969

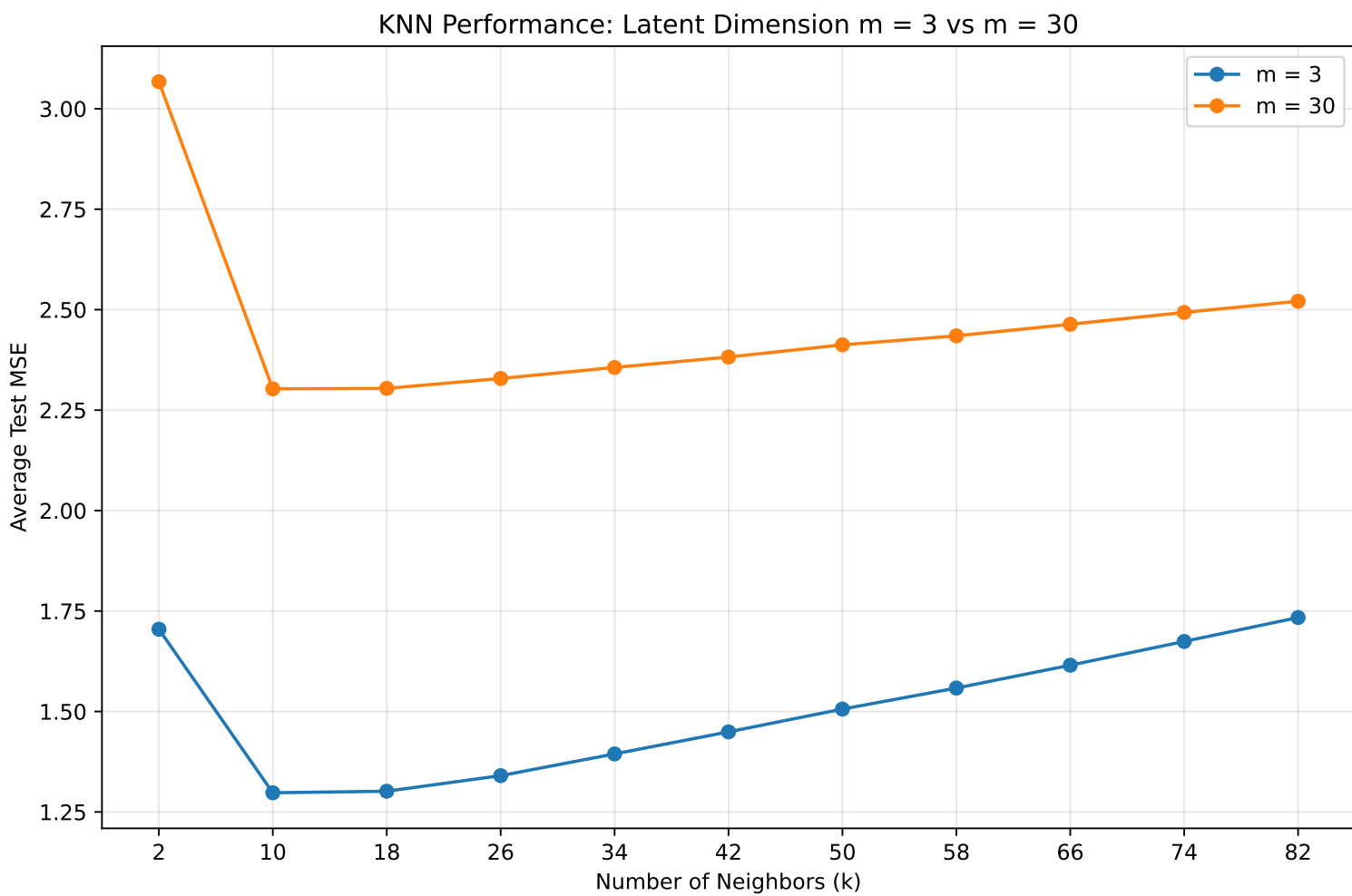


Figure 2: Average test MSE of KNN across k for $m = 3$ and $m = 30$ (50 repetitions).

The random seed used for the 50 repetitions was 5302. After averaging across the repetitions, $m = 3$ consistently had a lower test MSE than $m = 30$. The lowest average MSE occurred at $k = 10$ for both settings: approximately 1.298 for $m = 3$ and 2.303 for $m = 30$. The MSE initially decreases as k increases because averaging over more neighbors reduces variability. For larger k , the neighborhoods become too broad and the MSE begins to increase.

Part c

Even though both settings have 100 observed covariates and the response depends only on Z_1 and Z_2 , their underlying structure is different. With $m = 3$, most of the systematic variation in the 100 observed variables comes from only three latent variables. Therefore, Euclidean distance in X is more closely related to distance in a low-dimensional latent space, making it easier for KNN to find useful neighbors.

With $m = 30$, distance in X reflects variation from many more latent variables, most of which do not directly affect the response. This makes it harder for KNN to identify observations that are close specifically in Z_1 and Z_2 , which helps explain the higher MSE.

Finally, the noise η_i adds independent variation to all 100 observed covariates. Because of this noise, the observed X 's do not lie exactly in the m -dimensional space generated by AZ_i ; instead, they are scattered around it. Thus, the latent dimension and the amount of noise can matter for KNN even when the observed dimension is the same.

Question 4: Classify handwritten digits

Use `zip.train` and `zip.test` from the R package `ElemStatLearn`, as in the lecture. The package has been retired from the active CRAN repository. If needed, install the [archived package](#) into your usual R library, then load the two datasets. Python users can use the course copies of the same [training data](#) and [test data](#). These compressed CSV files have no headers and are included in the homework ZIP under `data/knn`. Run the Python code from the folder containing `data`.

Each image has $16 \times 16 = 256$ pixel values. The first column is the digit label and the remaining columns are the pixel values. Use the first 1,000 rows of `zip.train` and the first 500 rows of `zip.test`. Use the original pixel values without standardization, and do not include the digit label in a distance calculation.

For a test image $\mathbf{x}_0$ and a training image $\mathbf{x}_i$, the two distances are

$$d_E(\mathbf{x}_0, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{256} (x_{0j} - x_{ij})^2}, \quad d_M(\mathbf{x}_0, \mathbf{x}_i) = \sum_{j=1}^{256} |x_{0j} - x_{ij}|.$$

- a. Write a 5NN classification function that allows either Euclidean or Manhattan distance. For each test image, calculate its distances to the training images directly, find the five nearest, and return the most frequent digit label. Break distance ties by the smaller training row number and voting ties by the smallest digit. Do not use a built-in distance or KNN function for this part.
- b. Apply the classifier to all 500 test images using each distance. For each fit, report the test classification error and a 10×10 confusion matrix, with rows representing predicted digits and columns representing true digits. Keep all digit labels from 0 through 9 in the matrix, including any with a zero count. Which digits are most often confused with one another?
- c. For Euclidean distance, display three incorrectly classified test images, together with their true and predicted labels. Comment on whether the mistakes are visually understandable. Compare the overall results of the two distance measures. Can they have similar total error while making mistakes on different images? Use your predictions to support your answer.

Answer

Part a

Seed for Question 4: 5401
Training shape: (1000, 256)
Test shape: (500, 256)

A 5NN classifier was written by directly calculating either Euclidean or Manhattan distance from each test image to all training images. The five closest training images were used to classify each test image. Distance ties were broken by the smaller training row number, and voting ties were broken by choosing the smallest digit. No built-in KNN or distance function was used.

Part b

Classification Errors
Euclidean: 0.122000
Manhattan: 0.138000

Euclidean Confusion Matrix										
True \ Predicted	0	1	2	3	4	5	6	7	8	9
0	118	0	7	1	0	4	2	0	2	0
1	0	66	0	0	2	0	0	0	0	1
2	0	0	39	1	0	0	2	0	0	1
3	0	0	0	23	0	4	0	0	2	0
4	0	0	1	0	27	1	1	1	0	1
5	0	0	0	1	0	13	0	0	0	0

6	0	2	0	0	0	1	41	0	2	0
7	0	0	0	0	1	1	0	34	0	4
8	0	0	3	2	0	2	0	0	37	0
9	1	0	0	1	4	0	0	1	1	41

Manhattan Confusion Matrix

True \ Predicted	0	1	2	3	4	5	6	7	8	9
0	118	0	6	1	0	5	3	0	4	0
1	0	67	2	0	2	0	1	0	0	1
2	0	0	35	0	0	0	2	0	0	0
3	0	0	1	24	0	4	0	0	2	0
4	0	0	1	1	27	1	0	1	0	1
5	0	0	0	1	0	11	0	0	0	0
6	0	1	0	0	0	2	40	0	2	0
7	0	0	2	1	1	0	0	34	0	5
8	0	0	2	0	0	3	0	0	34	0
9	1	0	1	1	4	0	0	1	2	41

Most common Euclidean confusions:

True 2 predicted as 0: 7 times
 True 9 predicted as 7: 4 times
 True 5 predicted as 3: 4 times
 True 5 predicted as 0: 4 times
 True 4 predicted as 9: 4 times
 True 2 predicted as 8: 3 times
 True 8 predicted as 6: 2 times
 True 8 predicted as 3: 2 times
 True 8 predicted as 0: 2 times
 True 6 predicted as 2: 2 times

Most common Manhattan confusions:

True 2 predicted as 0: 6 times
 True 9 predicted as 7: 5 times
 True 5 predicted as 0: 5 times
 True 8 predicted as 0: 4 times
 True 5 predicted as 3: 4 times
 True 4 predicted as 9: 4 times
 True 6 predicted as 0: 3 times
 True 5 predicted as 8: 3 times
 True 8 predicted as 9: 2 times
 True 8 predicted as 6: 2 times

The test classification error was 0.122 (61/500) for Euclidean distance and 0.138 (69/500) for Manhattan distance, so Euclidean distance performed slightly better overall.

For Euclidean distance, the most common error was a true 2 classified as 0, which occurred 7 times. Other common confusions included 4 and 9, 7 and 9, and true 5s being classified as 0 or 3. Manhattan distance showed similar patterns, with 2 classified as 0 also being the most common mistake.

Part c

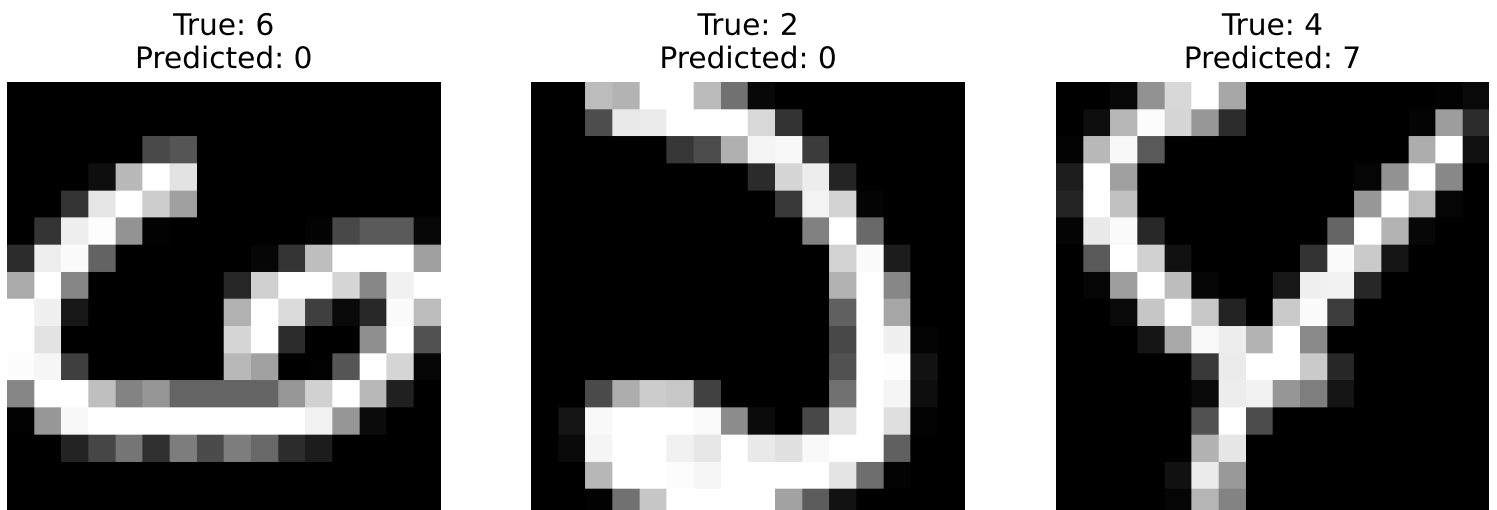


Figure 3: Three test images misclassified by Euclidean 5NN, with true and predicted labels.

Comparison of Euclidean and Manhattan mistakes
Both wrong: 58
Euclidean wrong, Manhattan correct: 3
Euclidean correct, Manhattan wrong: 11
Both correct: 428
Different predicted labels: 22

The three displayed Euclidean errors were a 6 predicted as 0, a 2 predicted as 0, and a 4 predicted as 7. These mistakes are visually understandable because the handwritten shapes share features with the predicted digits. For example, the 6 has a large rounded shape similar to a 0, while the 4 has strokes that can resemble a 7.

Overall, Euclidean distance performed slightly better than Manhattan distance, but the two methods did not make exactly the same mistakes. They gave different predictions for 22 images. There were 3 images that only Euclidean misclassified and 11 images that only Manhattan misclassified. This shows that two distance measures can have fairly similar overall error rates while still making mistakes on different individual images.

Question 5: Create a plot-formatting skill

Create a short `SKILL.md` file for improving the formatting of STAT 432 homework plots. You may use AI to help write it.

- Include a title, a description, and a few formatting rules. The description should limit the skill to STAT 432 homework plot improvements and require an explicit request to use it. Your rules should address margins, titles, axes, labels, colors, and sizes for plots produced in R or Python. Keep them portable: do not refer to specific homework questions.
- Ask your agent to install the skill. Explicitly call it to improve the plot from Question 1, then compare the original and revised versions. Do you like the changes? You may revise the skill and test it a couple more times, or uninstall it if it remains unhelpful.
- Append your final `SKILL.md` content to your submission and briefly explain your choices and experience.

Answer

Part a

The skill was saved as `stat432-plot-formatting/SKILL.md`:

```
---
name: stat432-plot-formatting
description: Improve the formatting of STAT 432 homework plots (R or Python) without changing the data, calculations, or
  ↳ meaning. Use only when the user explicitly asks to apply the STAT 432 plot-formatting skill.
disable-model-invocation: true
---
```

STAT 432 Plot Formatting

Description

Improve the appearance and readability of plots created for STAT 432 homework. Use this skill only when the user

- ↳ explicitly asks to apply the STAT 432 plot-formatting skill. It applies to plots produced in either R or Python and
- ↳ should improve formatting without changing the data, calculations, or meaning of the plot.

Formatting Rules

- **Margins:** Use balanced margins and enough spacing so titles, axis labels, tick labels, and legends are not clipped.
- **Titles:** Use a clear, concise title with a slightly larger font than the rest of the plot. Avoid unnecessary or
- ↳ overly long titles.
- **Axes:** Keep axes easy to read with sensible limits and tick spacing. Do not distort the scale or data.
- **Labels:** Give both axes descriptive labels, including units when relevant. Use readable font sizes.
- **Colors:** Use a small set of clearly distinguishable, readable colors. Keep colors consistent across related curves
- ↳ or categories and avoid unnecessary decoration.
- **Sizes:** Use a figure size suitable for a homework report. Make lines, points, text, and legends large enough to
- ↳ remain readable when the plot is included in the submitted document.

Preserve the statistical content of the original plot. Formatting changes should improve clarity and presentation rather

- ↳ than alter the analysis.

Part b

I installed the `stat432-plot-formatting` skill and explicitly applied it to the Question 1 plot. I prefer the revised version because the larger text, clearer axis labels, improved spacing, and distinct colors and marker shapes make the plot easier to read. The revised legend is also simpler, and the MSE curve is easier to distinguish from the bias and variance curves. The changes improved the presentation without changing the data or conclusions, so I kept the skill without further revisions.

Bias-variance trade-off of KNN at $\mathbf{x}_0$

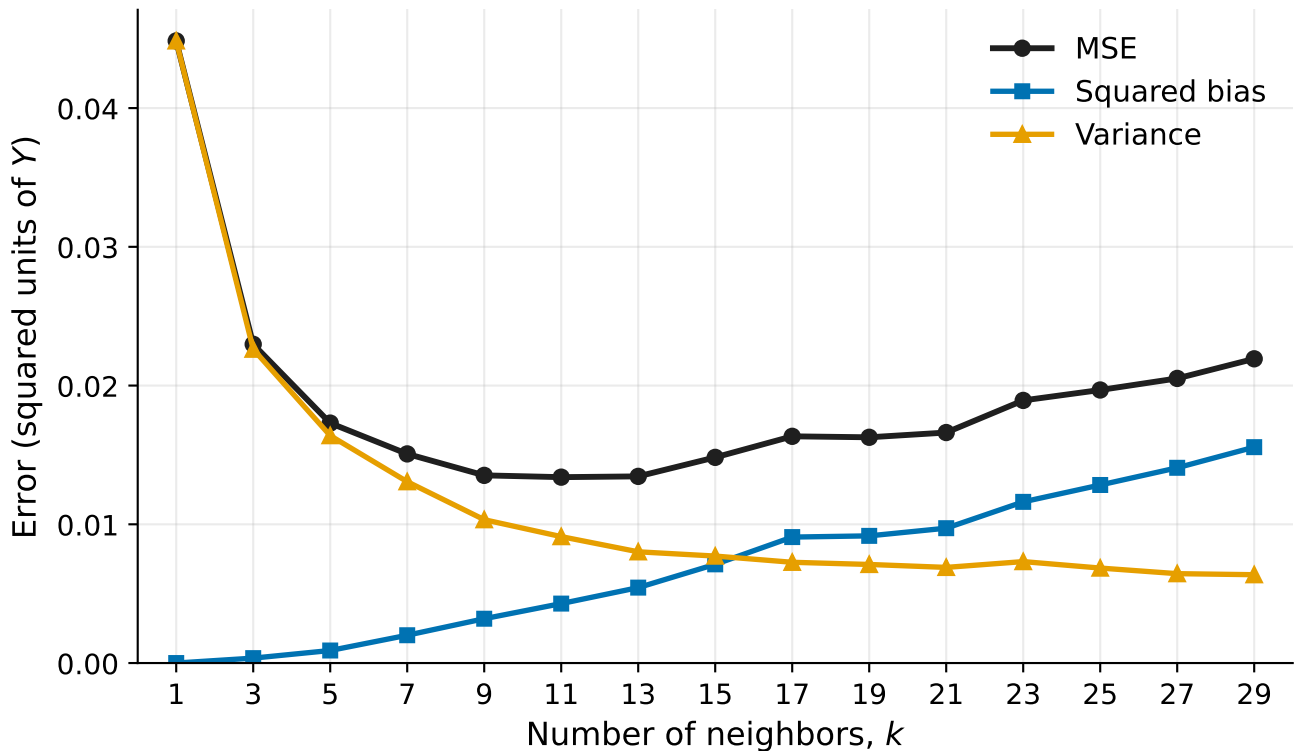


Figure 4: Question 1 plot after applying the `stat432-plot-formatting` skill (same data as the original Question 1 figure).

Part c

I chose these formatting rules to make STAT 432 homework plots clearer, more consistent, and easier to read without changing the statistical results. I focused on spacing, readable labels, appropriate figure sizes, and distinct colors so that the plots would look professional.

The skill was kept without further revisions after testing. Final `SKILL.md`:

```
---
name: stat432-plot-formatting
description: Improve the formatting of STAT 432 homework plots (R or Python) without changing the data, calculations, or
  ↳ meaning. Use only when the user explicitly asks to apply the STAT 432 plot-formatting skill.
disable-model-invocation: true
---
```

STAT 432 Plot Formatting

Description

Improve the appearance and readability of plots created for STAT 432 homework. Use this skill only when the user

- ↳ explicitly asks to apply the STAT 432 plot-formatting skill. It applies to plots produced in either R or Python and
- ↳ should improve formatting without changing the data, calculations, or meaning of the plot.

Formatting Rules

- **Margins:** Use balanced margins and enough spacing so titles, axis labels, tick labels, and legends are not clipped.
- **Titles:** Use a clear, concise title with a slightly larger font than the rest of the plot. Avoid unnecessary or
- ↳ overly long titles.
- **Axes:** Keep axes easy to read with sensible limits and tick spacing. Do not distort the scale or data.
- **Labels:** Give both axes descriptive labels, including units when relevant. Use readable font sizes.
- **Colors:** Use a small set of clearly distinguishable, readable colors. Keep colors consistent across related curves
- ↳ or categories and avoid unnecessary decoration.
- **Sizes:** Use a figure size suitable for a homework report. Make lines, points, text, and legends large enough to
- ↳ remain readable when the plot is included in the submitted document.

Preserve the statistical content of the original plot. Formatting changes should improve clarity and presentation rather

- ↳ than alter the analysis.